On Automatic Generation of Executable Domain Models for Domain-Driven Design

Van-Vinh Le*†, Nghia-Trong Be*, and Duc-Hanh Dang*
*Department of Software Engineering,
University of Engineering and Technology, Vietnam National University, Hanoi
†Department of Information Technology, Vinh University of Technology Education
Email: levanvinh@vuted.edu.vn, 20021400@vnu.edu.vn, hanhdd@vnu.edu.vn

***Abstract*—Domain-driven design (DDD) aims to iteratively develop software based on a realistic model of the problem domain. This approach requires a thorough understanding of the domain's requirements and ensuring that the implementation is technically feasible for deployment. Currently, such a domain model is often represented using programming languages such as Java and is manually defined by taking as input a specification in natural language and UML/OCL diagrams. It is very tedious to obtain a domain model in that way. This paper introduces a technique to automatically generate a domain model from a high-level specification in the form of UML/OCL diagrams. Specifically, we take as input the Activity diagram, which incorporates a Class diagram to describe the domain, and transforms it into an AGL$^+$ specification. The generated AGL$^+$ specification is executable based on the Java software framework jDomainApp. We develop tool support for our method using the Acceleo transformation language and illustrate our method with a case study.**

***Index Terms*—Domain-Driven Design, UML/OCL, Model Transformation.**

I. Introduction

Domain-driven design (DDD) [1] aims to iteratively develop software around a realistic model of the problem domain, ensuring both a thorough grasp of the domain requirements and technical feasibility for implementation. Currently, such a domain model is often represented using programming languages such as Java and is manually defined by taking as input a specification in natural language and UML/OCL diagrams. It is very tedious to obtain a domain model in that way. This introduces a need to transform such a high-level specification, e.g., with UML/OCL diagrams, into an executable domain model.

A high-level domain specification typically consists of two parts, one captures the structural view with domain concepts using Class diagrams, while the other represents domain behaviors through Activity diagrams. Recent research in DDD [2], [3] proposes annotation-based domain-specific languages (aDSLs) built upon object-oriented programming. These aDSLs aim to simplify the construction of domain models. In our recent work, we have defined a DSL named AGL$^+$ [4] to represent a unified domain model. This model incorporates behavioral aspects captured using Activity diagrams. To obtain an executable software version from such a domain model representation, two common approaches are considered. One involves embedding the implementation directly in the host programming language, incorporating other program concerns. The other, more indirect approach, entails utilizing model-driven techniques. This approach involves composing the domain model with other high-level concerns expressed using a general language such as UML and DSLs. The final program is then derived through model transformations, either model-to-model or model-to-text.

This paper focuses on defining a transformation from a high-level domain specification, which includes Activity and Class diagrams, to a corresponding AGL$^+$ specification. This transformation is implemented using the Acceleo transformation language [5]. The generated AGL$^+$ specification is subsequently integrated into the Java framework jDomainApp [6] to produce an executable software version.

In brief, this paper makes the following contributions:

- A technique to automatically generate an executable domain model in AGL$^+$ from a high-level domain specification with Activity and Class diagrams.
- Tool support provided for the proposed method, along with a case study to demonstrate its application.

The rest of the paper is organized as follows. Section II presents the motivating example and the technical background. Section III overviews our proposed approach. Section IV explains the Acceleo transformation from the high-level domain specification to a corresponding AGL$^+$ specification. Section V presents our tool support. Section VI reviews the related work. The paper is closed with a conclusion and an outlook on future work.

II. Motivating Example and Background

A. Motivating Example

We use the order management domain OrderMan [7, p. 396] as our motivating example. The OrderMan is a complex software model, which is adapted from the OMG/UML specification with an Activity diagram and a Class diagram together with OCL constraints. Figure 1 shows the domain model for OrderMan. In this Activity diagram, many kinds of `ControlNodes` are utilized

for the domain specification: A `DecisionNode` after the activity ReceivedOrder illustrates a branching based on two possible conditions: OrderRejected or OrderAccepted. FillOrder is followed by a `ForkNode` that passes control to both activity nodes SendInvoice and ShipOrder. The `JoinNode` indicates that the control will be passed to a `MergeNode` when both ShipOrder and AcceptPayment are completed. As the `MergeNode` will simply pass the token along, the EndOrder node will be executed.

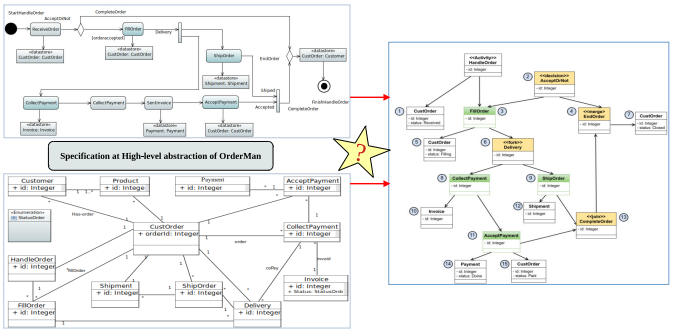

Fig. 1. Motivating Example

To derive an AGL$^+$ specification from a high-level domain specification, we follow these three steps [4]: Firstly, we start with a high-level domain specification comprising two parts: (1) UML Activity diagrams together with guards for coordinating nodes to capture the behavioral aspects of the domain and (2) the UML Class diagrams attached with OCL constraints to capture essential domain concepts. We could employ DCSL [8] to represent this part of the domain. Secondly, we convert the input domain specification into an AGL$^+$ specification, consisting of two parts: (1) the DCSL specification for the unified domain model [4]; and (2) the AGL specification for the activity graph configuration that corresponds to the input Activity diagram. Finally, we integrate the AGL$^+$ specification of the domain model into the framework JDomainApp [6] in order to obtain an executable software version.

The UML Activity diagram is often used to model the overall system behavior and to show the entire business process flow. Many approaches and methodologies have been proposed for the automatic source code generation from the UML Activity diagram [9]–[11]. In [10] the authors used a modified DFS (Depth-First Search) algorithm to generate a set of paths from the Activity diagram. For example, Figure 2 shows a newly created instance of the class `HandleOrder` of the OrderMan which is received by the action `ReceiveOrder` (the operation `initHandleOrder`() of the class `HandleOrder` is invoked). Subsequently, if the guard condition (`Self.rejectOrder`()) of the decision node is false (indicating that the order is not rejected), the flow proceeds to the next step: the action `FillOrder` (`Self.fillOrder`()). After that, the fork node splits the path of the control flow into two parallel tasks. If, however, the order is rejected, the flow directly proceeds to the action `EndOrder`. We defined the mapping using a `ControlNode` of the domain behavior patterns transformations into AGL.

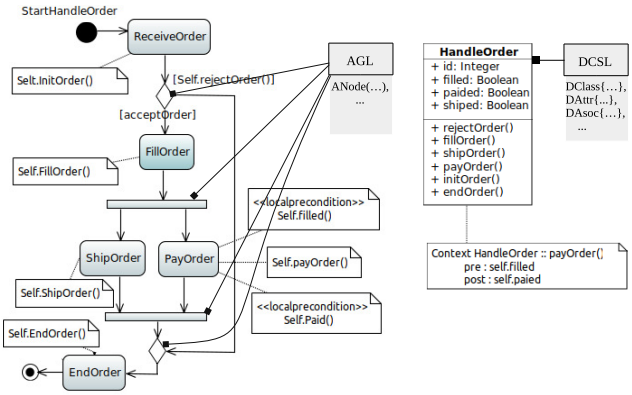

Fig. 2. The UML/OCL Activity and Class diagram of `HandleOrder` of OrderMan

A Class diagram [7] describes the structure of a system by showing the classes of the systems, their attributes, and the relationships among the classes. For UML Class diagrams, OCL is the language of choice for defining constraints going beyond simple multiplicity and type constraints in Figure 2. We transform a Class diagram into an equivalent DCSL, which takes an annotation-based form, specified in the OOPL's meta model.

B. Specifying Precisely Domain Models for a Domain-Driven Design

Domain-Driven Design (DDD)

Domain-Driven Design (DDD) [1], [12] aims to develop complex software. The DDD methodologies captures the domain requirements and is technically feasible for implementation. In practice, complex software requires a scalable architecture and a highly productive software development method. Within the DDD approach [1], the domain model tends to be the heart of software, the domain model would satisfy the domain requirements that are expressed in a ubiquitous language. This language is defined for stakeholders, including the domain experts and developers, in an iterative and agile process of eliciting the domain requirements. This work uses DDD to refer specifically to object-oriented and to construct DDD to generate software artifacts from a domain model. The domain model is feasible for implementation in a host OOPL [6].

A DCSL and AGL specification

Annotation-Based Domain Specific Language (aDSL) is coined in [13] as an attempt to formalise the notion of fragmentary, internal DSL [14] for the use of annotation to define DSLs. An aDSL is defined based on a set of meta-concepts that are common to two popular host OOPLs, the Java [15] and the C#. **Domain class specification language** (**DCSL**) [8] is a horizontal aDSL developed by us to express domain models.

In this work, we use feature of DCSL, which uses its meta-concepts model the domain-specific terms composed of the core OOPL meta-concepts and constraints. More specifically, the meta-concepts includes: **Domain Class**, **Domain Field**, **Associative Field** and **Domain Method** .

In our previous work [4], we proposed the Activity Graph Language (AGL) to incorporate the domain behaviors into the domain model for the composition of both structural and behavioral aspects of the domain. Each of the domain behaviors, described at high level using a UML Activity diagram and domain-model based statements, is translated into a specification that consists of two parts: (1) a part of the unified class model with new activity classes, and (2) the activity graph logic of the input activity and the mappings to connect the activity with the unified class model.

To create software artifacts following the DDD approach, we must combine DCSL and AGL specification (namely AGL$^+$ specification), which can be use to produce in a Java program.

The AGL$^+$ specification of OrderMan, resulting in a generated software

For AGL: The activity diagram specified by the AGL represents the `HandleOrder` class. Using the annotation mechanism in Java, the HandleOrder object can be treated as an AGraph object, allowing it to represent and manage the activity diagram. The AGraph object enables it to handle each of its ANodes. Listing 1 illustrates the annotation used to express the ANode w.r.t node 14 in the AGL specification of the `HandleOrder` activity class.

For DCSL: The class diagram specified by the DCSL for the class `HandleOrder` of OrderMan is shown in Figure 2. Listing 2 provides an example DCSL specification of `HandleOrder` written in Java which is specified with `DClass.serialisable` = `false`, the domain field name is immutable with `DAttr.mutable` = `false`.

Listing 1. The AGL specification of `HandleOrder` written in Java

```
/**Activity graph configuration in AGL */
@AGraph(nodes={...
  /* 14 */
  @ANode(label="14:Payment", zone="11:AcceptPayment",
  refCls=Payment.class, serviceCls=DataController.class,
  outNodes={"15:CustOrder"},
  actSeq={
    @MAct(actName=newObject, endStates={NewObject}),
    @MAct(actName=setDataFieldValues, attribNames={"invoice"},
    endStates={Created})
  }), ...
})
/**END: activity graph configuration */
```

Listing 2. The DCSL specification of `HandleOrder` written in Java

```
/** DCSL specification of \clazz{HandleOrder} written in Java*/
@DClass(serialisable=false, singleton=true)
public class HandleOrder {
  @DAttr(name = "id", id = true, auto = true, type = Type.
       Integer, length = 5,
  optional = false, mutable = false)
  private int id;
  private static int idCounter = 0;
  ...}
```

C. Research Question

To achieve this goal, we face the following primary challenge that motivate our work:

How can we obtain an AGL$^+$ specification by a model transformation from a specification of the domain model at high-level abstraction with UML Activity diagram and UML/OCL Class diagram?

III. Overview of Our Approach

This section explains our basic idea on transforming a high-level specification to an executable domain model for domain-driven design. Figure 3 overviews our proposed

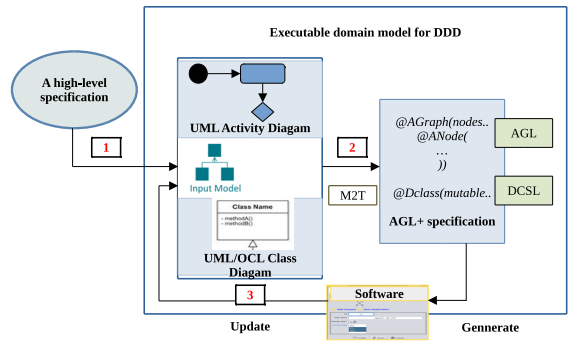

Fig. 3. Overview of Our Approach

technique. This technique conceptually consists of iteratively performing three steps.

Step 1 (The construct domain model takes input): We take a specification with UML/OCL Class Diagram (CD) and Activity Diagram (AD) as input. To achieve target of AGL$^+$ specification, we transform the CD input to the DCSL specification and the AD input to the AGL specification. A detailed technical specification on how to generate AGL$^+$ specification from high-level specification AD and CD is in Section IV. Here, we consider the AGL$^+$ specification as an extended domain model in a module-based software architecture [6]. It is expressed in AGL and DCSL as an extension of the conventional DDD's domain model [1] with the executable domain model.

Step 2 (The AD2AGL transformation): This step transforms the input AD specification into an AGL specification and then composes it with DCSL specification, which is implemented in DDD as a GUI- and module-based software. This software is presented to the domain expert in order to get feedback.

Step 3 (The update and the cycle): If there is feedback, then the input domain model will be updated and the cycle continues. If, on the other hand, the domain expert is satisfied with the models; then the cycle ends.

IV. Transforming ADs to the AGL Specification

A. Algorithm AD2AGL

This section presents an algorithm to generate an AGL$^+$ specification of the domain model from an UML Activity and Class diagram. We call it the AD2AGL algorithm.

Algorithm 1 AD2AGL takes as input AD (the UML Activity diagram that captures the system behavior) and CD (the UML/OCL Class diagram that specifies the domain model), and produces as output DM (an AGL^+ specification of the domain model with annotated and realized in Java; e.g. @AGraph, @ANode, @MAct, @DClass, @DAttr, etc.).

Algorithm 1 Generating an AGL specification from an UML Activity diagram

Require:

- *AD*: the UML Activity diagram to capture the system behavior.
- *CD*: the UML Class diagram to specify the domain model.

Ensure:

- *DM*: an AGL^+ specification of the domain model

Initialization: $AG \Leftarrow$ the activity graph w.r.t. AD; $curNode \Leftarrow$ the initial nodel of AG
while $curNode$ is not the final node of AG **do**
  if $curNode.next$ is a Decision node **then**
    $genDecisionNode(curNode, DM)$
  else if $curNode.next$ is a Sequential node **then**
    $genSequentialNode(curNode, DM)$
  else if $curNode.next$ is a Fork node **then**
    $genForkNode(curNode, DM)$
  else if $curNode.next$ is a Merge node **then**
    $genMergeNode(curNode, DM)$
  else if $curNode.next$ is a Join node **then**
    $genJoinNode(curNode, DM)$
for each $class \in CD$ **do**
  $genDCSL(class, DM)$

First, we initialize an empty DM to contain an AGL^+ specification of the domain model and point to the initial node of AD.

Second, it uses the Depth-First Search algorithm on the AD to visit all nodes in the activity graph, and an AGL specification will be added whenever a new action node is visited. As per UML [7], an activity diagram is considered a graph. An Activity Graph (AG) [10] is a hextuple containing nodes N, edges E, events eve, guard conditions gua, local variables var, and set of objects obj. A node can be of two types: *ActionNode* and *ControlNode*. The *ActionNode* includes *action node, acceptEvent node,* and *sendSignal node*. The *ControlNode* includes *initial node, final node, sequential node, decision node, merge node, fork* and *join node* and for more information, we refer the reader to Appendix A of the technical report [16] accompanying this paper.

Third, it checks each node in the AD, including all action nodes and control nodes, and calls sub-functions for the respective cases to generate AGL specification. When visiting the current node and identifying the next node from the current node, it checks the next node and calls the functions for generating AGL with input parameters corresponding to the current node ($curNode$) and updated the AGL specification of the domain model (DM). The functions are: *genSequentialNode(curNode,DM)*, *genDecisionNode(curNode,DM)*, *genForkNode(curNode,DM)*, *genMergeNode(curNode,DM)* and *genJoinNode(curNode,DM)*. The function adds an AGL specification according to the annotation-based textual concrete syntax model for AGL that is defined in [4].

Fourth, it visits the next node and repeats step third until all nodes have been visited (meet *finalNode*).

Fifth, generate DCSL from CD by traversing all classes on CD and updating the DCSL specification in the domain model (DM).

Due to the length constraint of this paper, we will only explain the function of the decision node in *genDecisionNode(curNode,DM)* and its specification in AD in this paper. For more information on the other functions (*genSequentialNode(curNode,DM)*, *genForkNode(curNode,DM)*, *genMergeNode(curNode,DM)* and *genJoinNode(curNode,DM)*) and their node specifications in AD and the AGL specification updated to DM, please refer to the accompanying technical report [16].

In the function *genDecisionNode(curNode,DM)*, which takes the input e_n - the current node, which has a next decision node named e_d in AD, represented in Figure 4.

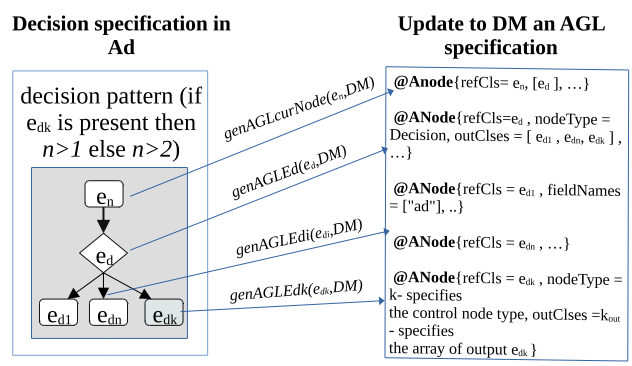

Fig. 4. The function for the decision node updates to DM an AGL

The function *genDCSL(class,DM)* generates DCSL from CD (DCSL specification according to the annotation-based approach is defined in [8]) by traversing all classes on CD and updating the DCSL specification in the domain model (DM).

B. Rules (templates in Acceleo)

To implement the code generator, the Model-to-Text transformation (M2T) is needed to generate the AGL^+. We define the rules (templates in Acceleo) to transform from the high-level specification AD and CD into the AGL^+.

With input as domain model in AD (the file **.uml*), the Algorithm 1 uses a repeat statement in Acceleo to read all Nodes (*ActivityNode*) in AD and uses the *oclIsKindOf()* method to check the type of *ActivityNode*: *DecisionNode, ForkNode, MergeNode, JoinNode* and *OpaqueAction* are

considered *SequentialNodes*, which will then call the corresponding generate AD to AGL function to update the DM, is shown in Figure 5.

```
generateAD2AGL.mtl ×
1  [module generateAD2AGL('http://www.eclipse.org/uml2/5.0.0/UML')]
2  [comment]--------generate AGL ----------------------[/comment]
3  [template public generateElementAGL(aModel : Model)]
4  [file (aModel.name.concat('.java'), false, 'UTF-8')]
5  /**Activity graph configuration*/
6  @AGraph(nodes={
7      [let nodeElements : Sequence(ActivityNode) =
8                  packagedElement->select(e | e.oclIsKindOf(Activity))
9                  ->collect(e | e.oclAsType(Activity).node)
10                 ->sortedBy(n | n.name)]
11     [for (aActivityNode : ActivityNode | nodeElements)]
12                 [let nextNode: ActivityNode=aActivityNode.outgoing->
13                                                  any(true).target]
14                     [if (nextNode.oclIsKindOf(OpaqueAction))]
15                         [callSequentialAGL(aActivityNode)/]
16                     [/if]
17                     [if (nextNode.oclIsKindOf(DecisionNode))]
18                         [callDecisionAGL(aActivityNode)/]
19                     [/if]
20                     [if (nextNode.oclIsKindOf(ForkNode))]
21                         [callForkAGL(aActivityNode)/]
22                     [/if]
23                     [if (nextNode.oclIsKindOf(MergeNode))]
24                         [callMergeAGL(aActivityNode)/]
25                     [/if]
26                     [if (nextNode.oclIsKindOf(JoinNode))]
27                         [callJoinAGL(aActivityNode)/]
28                     [/if]
29                 [/let]
30     [/for]
31     [/let]
```

Updates to DM an AGL with Sequential pattern

Updates to DM an AGL with Decision pattern

Updates to DM an AGL with Fork pattern

Updates to DM an AGL with Merge pattern

Updates to DM an AGL with Join pattern

Fig. 5. The rules (templates in Acceleo) to transform from the high-level specification AD to AGL

The template in Acceleo to transform a *Decisional* node in AD, with the function *genDecisionNode(curNode)*, consists of the following steps:

1) Representing the template configured unified model of the pattern as AGL$^+$;
2) Reading all the outgoing edges of the *DecisionNode* and matching them with the template to the activity in order to define `ANodes`. Based on the guards condition, the reference from `ANodes` to domain classes establishes a mapping between the activity and the unified class model;
3) Expressing the generic behavior of the domain module. This ensures that the execution state of the activity is synchronized with the current state of the unified class model.

The template in Acceleo to transform from the UML/OCL Class diagram to DCSL specification using the function *genDCSL(model:Model)*. We design an input model that consists of classes, properties, associations and OCLs in CD, the function consists of the following steps:

1) Representing the template DCSL of the AGL$^+$ specification;
2) Reading all the `Classes` in the CD specification and matching them with the template to the method of `Class`. Based on the constraints on classes, fields, and methods on the Class diagram, mapping them to the DCSL template of the AGL$^+$ specification;
3) Using Query templates to extract information from the model easily for mapping *DClass, DAttr, DOpt* and *DAssoc* to update the AGL$^+$ specification of the domain model.

Listing 3 provides examples of the Query templates for **@MAct** and **@DClass**. The generated code is based on the Query templates.

Listing 3. The template generate the @*MAct* and @*DClass*

```
  /**actName=newObject,pstStates={Created}*/
[query public genMActcreateObject() :
Set(String)='@MAct(actName=newObject,pstStates=[Created])'/]
/** $@$DClass(serialisable=true, singleton = true)*/
[query public genDClass(serialisable: String,singleton: String) :
Set(String)='@DClass(serialisable='.concat(serialisable).concat('
     ,singleton=').concat(singleton).concat(')') /]
```

V. Tool Support

In this section, we present a case study of order products management OrderMan to illustrate our technique of transforming the high-level specification AD of OrderMan to AGL, using Acceleo. Acceleo provides powerful

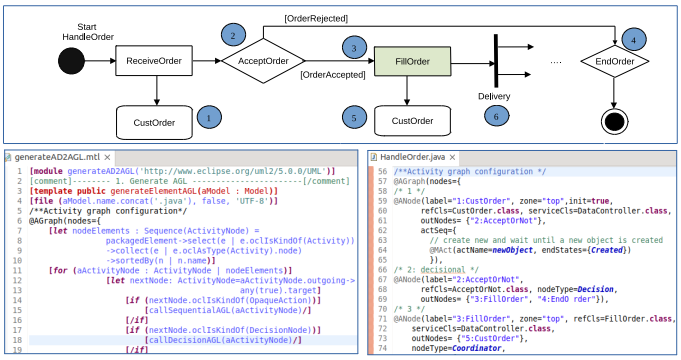

Fig. 6. Use Acceleo to transform AD of OrderMan to AGL, `HandleOrder` realized in Java

tooling such as an editor with syntax highlighting, error detection, code completion, refactoring, debugger, profiler, and a traceability API that allows tracing model elements to the generated code and vice versa [5]. Figure 6 shows the AD of OrderMan at the top, which is the input model of our method. The bottom left shows the Acceleo code (template) that performs the transformation to the **HandleOrder** code, the output of our method is the AGL specification on the bottom right.

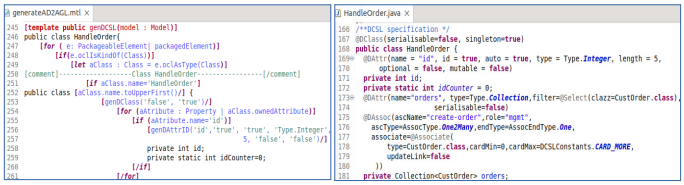

Fig. 7. Use Acceleo to transform the CD of the OrderMan to the DCSL

We also use Acceleo to transform the high-level UML/OCL Class diagram to a DCSL specification. In Figure 7, the Acceleo code (template) to generate DCSL is shown on the left, and the resulting DCSL is shown on the right. We refer the reader to Appendix B of the technical report [16] accompanying this paper. Next, we compose the AGL with the DCSL to form the complete AGL$^+$ specification, which is used as input to automatically generate and implement our method, using a supporting tool built on jDomainApp, a Java software framework.

VI. Related Work

We position our work in the automatic generation AGL$^+$ specification by a model transformation from a specification of the domain model at high-level abstraction with UML AD and UML/OCL CD and executable domain model for DDD. Existing proposals for converting the behavioral models to the implementation code [9], such as the algorithm for code generation from activity and sequence diagrams [10], the authors used a modified DFS algorithm to generate a set of paths from the AD. Another proposal is the solution for automating transformation from CIM (Computing Independent Model) level business-based to the PIM (Platform Independent Model) level web-based [11]. In this paper, we used DFS algorithm to transformation AD into the PIM to simplify software development by transforming at high-level specification (UML behavior models) into source code.

The work [4] proposes to incorporate the domain behaviors (that can be captured by UML Activity diagrams) into a domain model for a composition of both structural and behavioral aspects of the domain and extend a domain definition language like DCSL with new constructs to represent such domain behaviors with the abstract and concrete syntax of the AGL. The concrete syntax of the AGL is represented using programming languages such as Java and is manually defined by taking as input a specification in natural language and UML/OCL diagrams. It serves as the basis for us to build an algorithm transformation from a specification of the domain model at high-level abstraction, with UML/OCL diagrams.

To execute domain model for DDD, Evans proposed methodologies in the book [1] that capture the domain requirements and are technically feasible for implementation. The work [8] proposes an annotation-based DSL named DCSL for designing the domain model, and the work [6] proposes a generative software module development method for DDD, with a module-based software architecture and a generative technique for module configuration. The authors of these works have not yet proposed to automatically generate DCSL specification from at high-level abstraction (with UML Class and Activity diagrams). In this paper, we look at generating DCSL specification at high-level abstract and creating software artifacts following the DDD approach from combining DCSL and AGL specification into AGL$^+$ specification automatically.

VII. Conclusion and future work

This paper has introduced a technique for transforming a high-level domain specification, which includes UML/OCL diagrams, into an executable domain model in AGL$^+$. The resulting specification can be integrated into the Java framework jDomainApp for an executable software version. We have developed a supporting tool using the Acceleo transformation language and have demonstrated the method's effectiveness through a case study.

As part of our future work, we plan to focus on the following points:

- Defining a mechanism to ensure that the input Activity diagram specification is a well-formed combination of domain behavior patterns
- The current DCSL specification is partially generated and manually written in Java. Our future work aims to fully automate its generation from the input UML/OCL diagrams. The composition of DCSL and AGL should also undergo automation.
- Developing an Eclipse plug-in for our method that enables the automatic generation of a complete AGL$^+$ specification from a requirements specification.

References

[1] E. Evans, Domain-driven design: tackling complexity in the heart of software, Addison-Wesley Professional, 2004.

[2] Dan Haywood, Apache Isis - Developing Domain-driven Java Apps, Methods & Tools: Practical knowledge source for software development professionals 21 (2) (2013) 40–59.

[3] J. Paniza, Learn OpenXava by Example, CreateSpace, Paramount, CA, 2011.

[4] D.-H. Dang, D. M. Le, V.-V. Le, Agl: Incorporating behavioral aspects into domain-driven design, Information and Software Technology 163 (2023) 107284. doi:https://doi.org/10.1016/j.infsof.2023.107284.

[5] M. Brambilla, J. Cabot, M. Wimmer, Model-driven software engineering in practice, Synthesis lectures on software engineering 3 (1) (2017) 1–207.

[6] D. M. Le, D.-H. Dang, V.-H. Nguyen, Generative software module development for domain-driven design with annotation-based domain specific language, Information and Software Technology 120 (2020) 106239.

[7] OMG, Uml 2.5.1 uml-unified modeling language (2017).

[8] D. M. Le, D.-H. Dang, V.-H. Nguyen, On domain driven design using annotation-based domain specific language, Computer Languages, Systems & Structures 54 (2018) 199–235.

[9] E. Sunitha, P. Samuel, Translation of behavioral models to source code, in: 2012 12th International Conference on Intelligent Systems Design and Applications (ISDA), 2012, pp. 598–603. doi:10.1109/ISDA.2012.6416605.

[10] S. E. Viswanathan, P. Samuel, Automatic code generation using unified modeling language activity and sequence models, Iet Software 10 (6) (2016) 164–172.

[11] Y. Rhazali, Y. Hadi, I. Chana, M. Lahmer, A. Rhattoy, A model transformation in model driven architecture from business model to web model, IAENG International Journal of Computer Science 45 (1) (2018) 104–117.

[12] V. Vernon, Implementing domain-driven design, Addison-Wesley, 2013.

[13] M. Nosál, M. Sulír, J. Juhár, Language composition using source code annotations, Computer Science and Information Systems 13 (3) (2016) 707–729.

[14] M. Fowler, D.-S. Languages, Addison-wesley professional (2010).

[15] J. Gosling, The java language specification: Java se 8 edition, (No Title) (2015).

[16] V.-V. Le, N.-T. Be, D.-H. Dang, On Transforming a High-Level Specification to an Executable Domain Model for Domain-Driven Design, Tech. rep., VNU University of Engineering and Technology, Vietnam (2023).
URL `https://tinyurl.com/AD2AGLTechnical`